

INF.04 — Przygotowanie do egzaminu

SORTOWANIE

Algorytmy sortowania krok po kroku

Bubble Sort • Selection Sort • Insertion Sort • Merge Sort • Quick Sort

1. Wstęp.

1.1. Dlaczego sortowanie jest ważne?

- Posortowane dane łatwiej przeszukiwać — zamiast przeglądać wszystko po kolei, możemy używać szybszych metod (np. wyszukiwanie binarne).
- Sortowanie jest podstawą wielu innych algorytmów.
- W codziennym życiu sortowanie jest wszędzie: listy kontaktów w telefonie, wyniki wyszukiwania, zakupy sortowane po cenie.

1.2. Jak mierzymy efektywność algorytmów sortowania?

Każdy algorytm sortowania można ocenić pod kątem dwóch rzeczy:

Kryterium	Co oznacza	Przykład
Złożoność czasowa	Ile operacji wykonuje algorytm	$O(n^2)$ oznacza, że dla 100 elementów może być 10 000 operacji
Złożoność pamięciowa	Ile dodatkowej pamięci potrzebuje	$O(1)$ oznacza, że nie potrzebuje prawie żadnej dodatkowej pamięci
Stabilność	Czy zachowuje kolejność równych elementów	Ważne np. przy sortowaniu listy osób po nazwisku

i Notacja dużego O — wyjaśnienie prostymi słowami

$O(1)$ — czas stały: nie zależy od liczby elementów (błyskawiczne)

$O(n)$ — czas liniowy: podwójnie więcej danych = dwukrotnie więcej czasu

$O(n \log n)$ — czas logarytmiczny: dobre algorytmy sortowania

$O(n^2)$ — czas kwadratowy: podwójnie więcej danych = 4 razy więcej czasu

2. Sortowanie bąbelkowe (Bubble Sort)

2.1. Idea algorytmu

Sortowanie bąbelkowe to najprostszy algorytm sortowania. Jego nazwa pochodzi od zachowania algorytmu — duże wartości wypływają na koniec tablicy jak bąbelki powietrza w wodzie.

Zasada jest prosta:

Przechodzimy przez tablicę wielokrotnie. Za każdym razem porównujemy dwa sąsiednie elementy. Jeśli są w złej kolejności — zamieniamy je miejscami. Powtarzamy to tak długo, aż żadna zamiana nie jest już potrzebna.

Analogia z życia

Wyobraź sobie, że stoisz w kolejce 5 osób ustawionych według wzrostu, ale losowo. Porównujesz osobę 1 z osobą 2 — jeśli pierwsza jest wyższa, zamieniasz miejsca. Potem porównujesz osobę 2 z osobą 3, potem 3 z 4, potem 4 z 5. Po pierwszym przejściu NAJWYŻSZA osoba stoi na końcu — to był pierwszy 'bąbel'. Powtarzasz to od początku, ale już bez ostatniej osoby (ona już jest na miejscu). Kontynuujesz, aż wszyscy stoją w odpowiedniej kolejności.

2.2. Przykład krok po kroku

Mamy tablicę: [5, 3, 8, 1, 4]. Chcemy ją posortować rosnąco.

PRZEJŚCIE 1 (i = 0):

Krok 0	5	3	8	1	4
indeks	0	1	2	3	4

Porównujemy indeks 0 i 1: $5 > 3 \rightarrow$ zamieniamy miejscami.

Po zamianie	3	5	8	1	4
indeks	0	1	2	3	4

Porównujemy indeks 1 i 2: $5 < 8 \rightarrow$ bez zamiany.

Bez zmian	3	5	8	1	4
indeks	0	1	2	3	4

Porównujemy indeks 2 i 3: $8 > 1 \rightarrow$ zamieniamy miejscami.

Po zamianie	3	5	1	8	4
indeks	0	1	2	3	4

Porównujemy indeks 3 i 4: $8 > 4 \rightarrow$ zamieniamy miejscami.

Po zamianie (koniec przejścia 1)	3	5	1	4	8
indeks	0	1	2	3	4

✓ Po przejściu 1

Tablica: [3, 5, 1, 4, 8]

Element 8 jest już na swoim miejscu — największy trafił na koniec!

W następnym przejściu nie musimy sprawdzać ostatniego elementu.

PRZEJŚCIE 2 ($i = 1$) — porównujemy tylko pierwszych 4 elementów:

Przed przejściem 2	3	5	1	4	8
indeks	0	1	2	3	4

$3 < 5 \rightarrow$ bez zamiany. $5 > 1 \rightarrow$ zamiana. $5 > 4 \rightarrow$ zamiana.

Po przejściu 2	3	1	4	5	8
indeks	0	1	2	3	4

PRZEJŚCIE 3 ($i = 2$):

$3 > 1 \rightarrow$ zamiana. $3 < 4 \rightarrow$ bez zamiany.

Po przejściu 3	1	3	4	5	8
indeks	0	1	2	3	4

PRZEJŚCIE 4 ($i = 3$):

$1 < 3 \rightarrow$ bez zamiany. Brak zamian = tablica już posortowana!

Wynik końcowy ✓	1	3	4	5	8
indeks	0	1	2	3	4

2.3. Kod w języku Java

KOD JAVA — Bubble Sort

```
public static void bubbleSort(int[] tab) {
    int n = tab.length;
```

```

    for (int i = 0; i < n - 1; i++) {           // n-1 przejść
        for (int j = 0; j < n - 1 - i; j++) {   // każde przejście krócej o
1
            if (tab[j] > tab[j + 1]) {         // zła kolejność?
                // zamiana sąsiednich elementów
                int temp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = temp;
            }
        }
    }
}

// Przykład użycia:
int[] dane = {5, 3, 8, 1, 4};
bubbleSort(dane);
// Wynik: [1, 3, 4, 5, 8]

```

2.4. Złożoność i cechy

Cecha	Wartość	Wyjaśnienie
Złożoność czasowa (śr.)	$O(n^2)$	Dwie zagnieżdżone pętle
Złożoność czasowa (naj.)	$O(n)$	Gdy tablica już posortowana
Złożoność pamięciowa	$O(1)$	Nie potrzebuje dodatkowej tablicy
Stabilność	TAK — stabilny	Równe elementy zachowują kolejność
Zastosowanie	Tablice małe/edukacja	Zbyt wolny dla dużych danych

Zapamiętaj na egzamin!

Bubble Sort: $O(n^2)$ — dwie zagnieżdżone pętle, $n-1$ przejść
 Po każdym przejściu jeden element więcej jest na swoim miejscu (od końca)
 Jest stabilny — równe elementy nie zmieniają kolejności względem siebie
 Jego zaletą jest prostota, wadą — wolność przy dużych danych